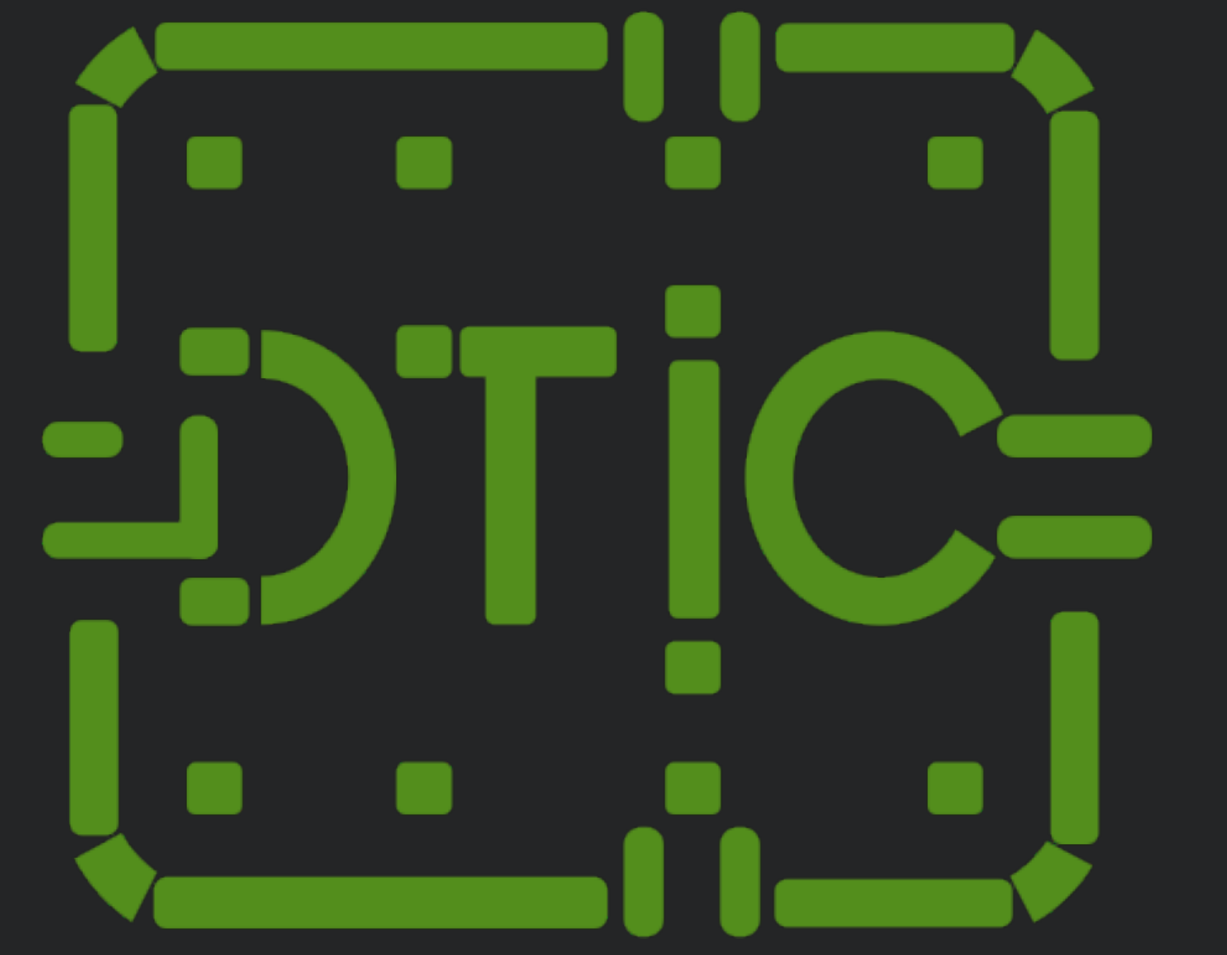




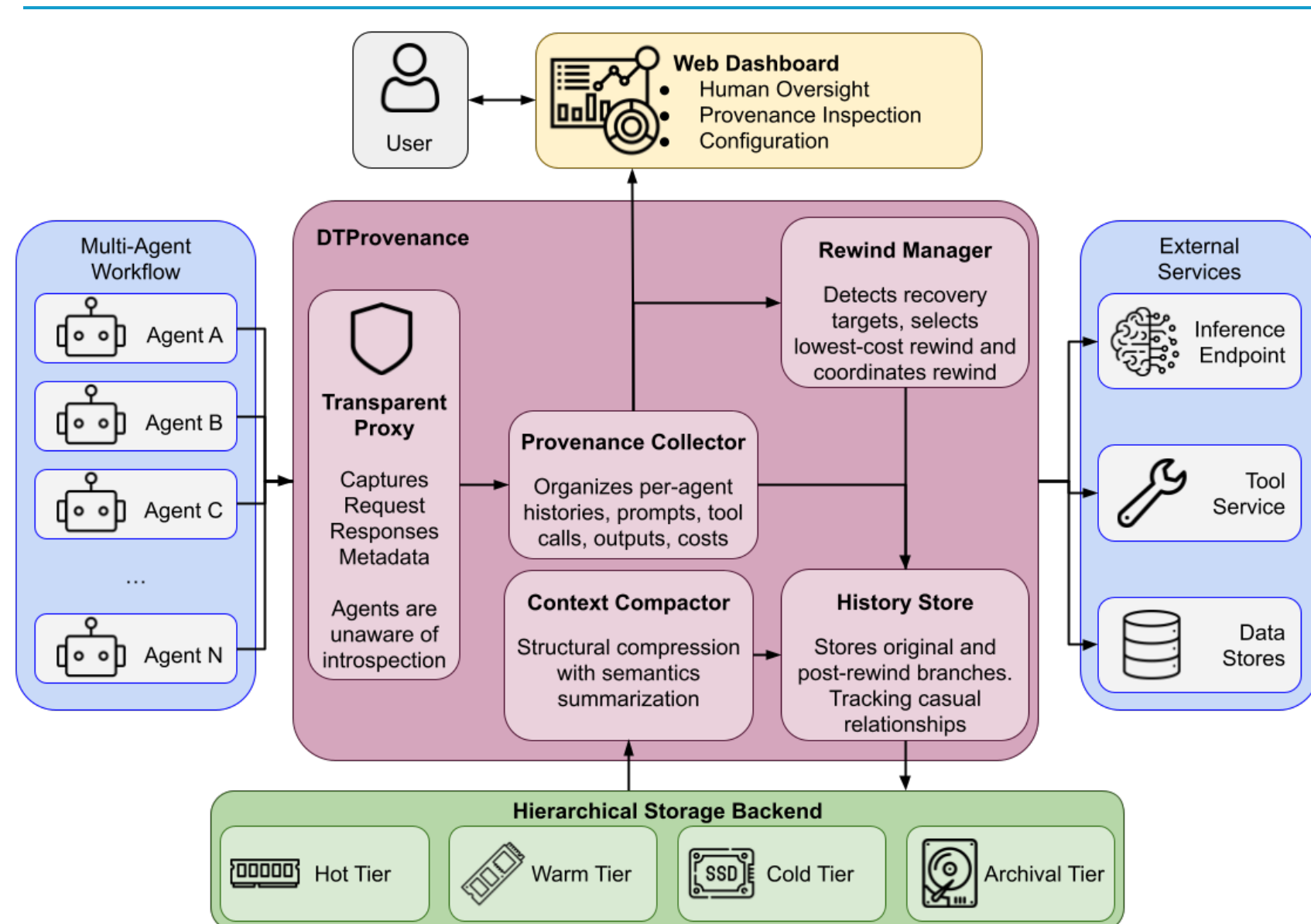
Introduction

- As Large Language Models (LLMs) improve their performance, multi-agent systems have been utilized more frequently in scientific workflows
- Failure is an inevitable problem in LLM systems, as hallucinations and unintended side effects can propagate throughout an entire workflow.
- Existing recovery systems focus on checkpointing and rewind for single agent systems, and fail to leverage advanced provenance information.
- Problem Statement:** There is a need for a system that can manage context and provide rewind for complex agentic workflows by leveraging the advanced lineage of provenance systems

Contributions include:

- Provenance Collection** framework that captures multi-agent interactions through various interfaces on heterogeneous data stores.
- Feedback and Recovery** mechanism that enables cross-agent context rewind with rich provenance data for error recovery.
- Context Management** system that preserves important context for the purposes of agent behavior introspection.

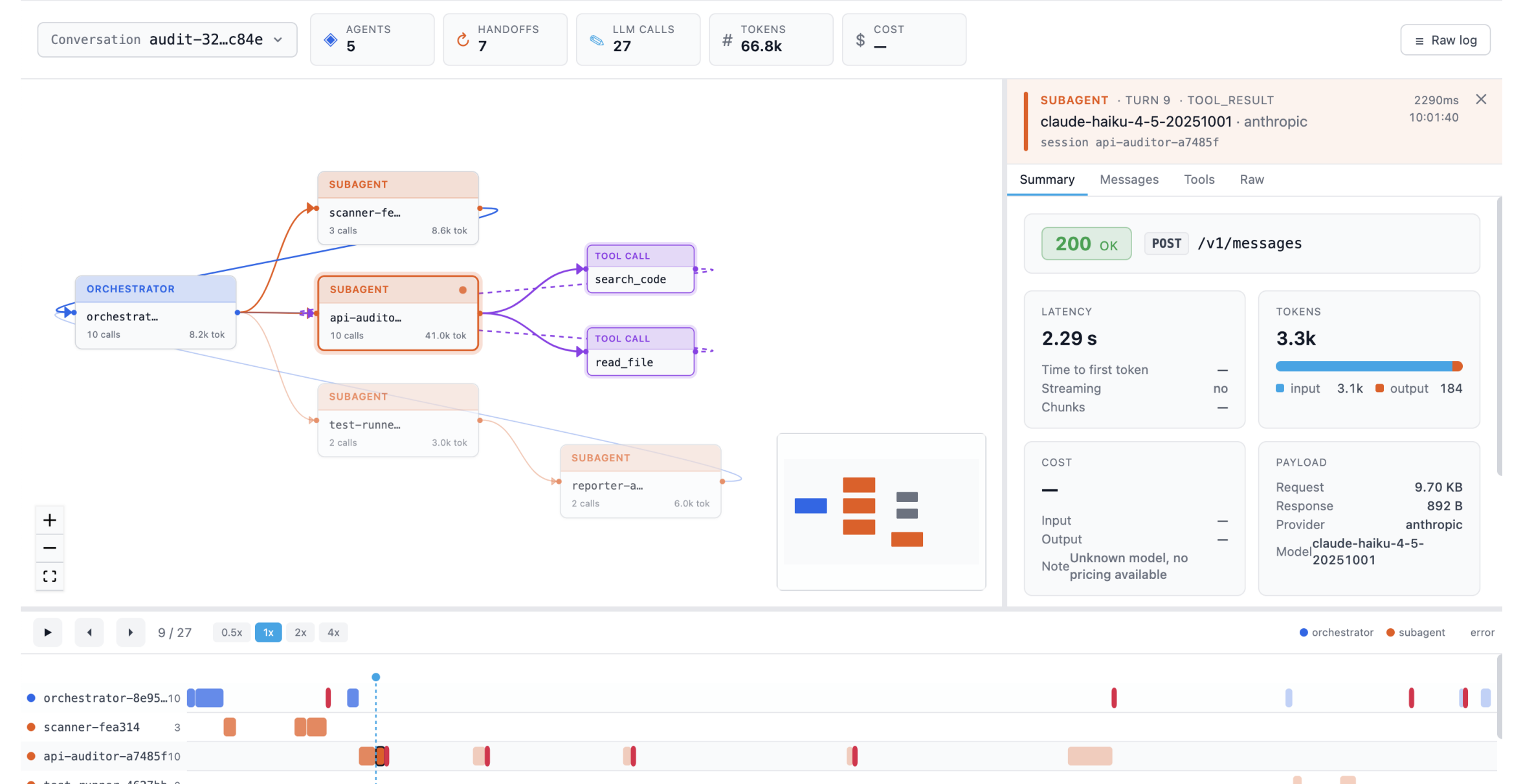
Approach



- General workflow involves capturing agent interactions through a Flask-based interception layer
- Hierarchical storage is provided by the IOWarp distributed task system

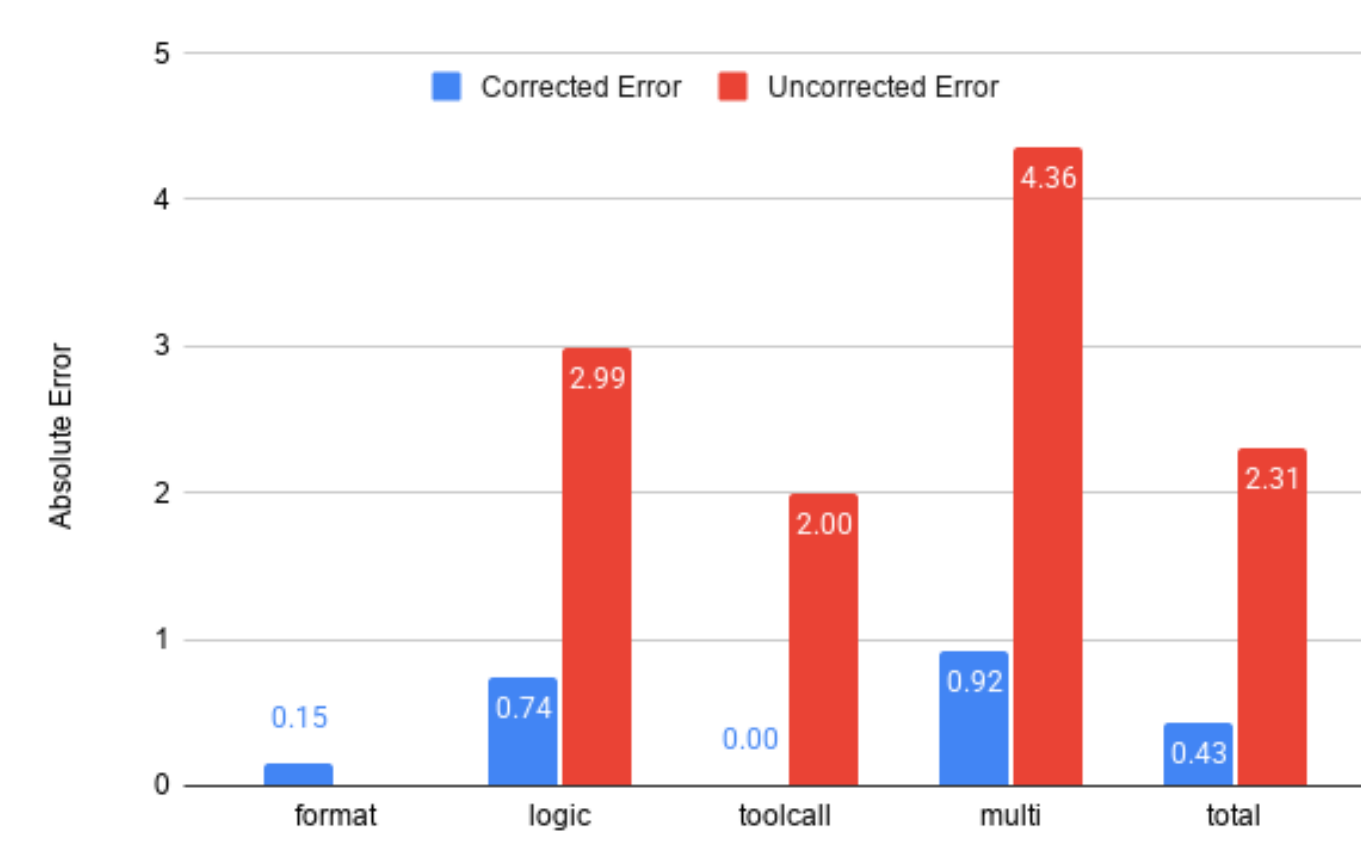
- Provenance Collector** tracks information and statistics on all agent interactions, including tool calls, inter-agent messaging, queries, system prompts, etc.
- Rewind Manager** permits for checkpoints to be restored from past context. This information is pulled from the hierarchical storage and triggered when either a human or an agent notices a mistake.
- Context Compactor** determines when an agent would receive more context than it can handle and creates a summary that maintains the core features including the query.
- History Store** permits metastats about provenance information to be tracked. Can be useful when performing context compaction as important history should be maintained.

Provenance Graph Demonstration



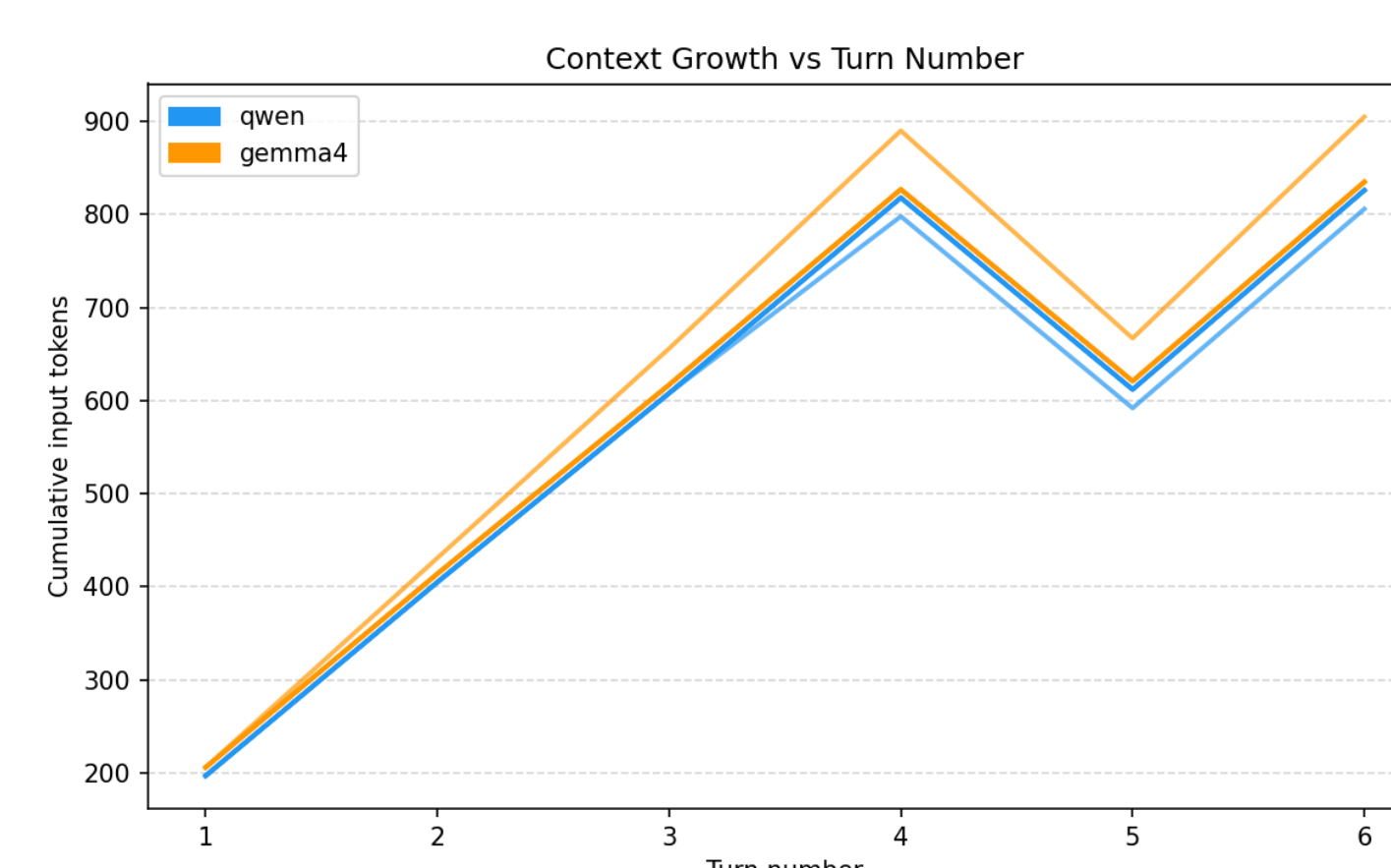
AgentRecall provenance graphs capture various statistics including context size, token input/output, token cost, call graphs.

Rollback Accuracy Benefits



- Tool call errors tend to be easier to fix, while logical and formatting errors tend to be difficult to fix
- We find an average error decrease of 84%

Context Simplification



- AgentRecall has configurable context compaction to match agent specifications
- Here you can see context compaction triggering to maintain a 1024 token context window

Conclusion

- AgentRecall provenance graphs capture information about various statistics to create a call graph
- Preliminary work shows rollback can contribute accuracy improvements up to 84% over baselines
- Rollback accuracy improvements are dependent on the type of error, with tool call errors being significantly easier to fix
- Context compaction enables AgentRecall to keep the system prompts it sends to the users brief

Acknowledgments

This material is based upon work supported in part by the National Science Foundation (NSF), Division of Computer and Network Systems (CISE/CNS), under Grant CCRI-CISE 2346504, Grant CSSI-2411386, and the Department of Energy (DOE), Office of Advance Scientific Computing Research (ASCR), under Grant DE-AC02-06CH11357. This research used the DeltaAI system at the National Center for Supercomputing Applications through allocation CIS250329 from the ACCESS program, supported by NSF award OAC 2320345.